

EXPERIMENT - 1

AIM - Create a class Box that uses a parameterized constructor to initialize the dimensions of a box. The dimensions of the Box are width, height, depth. The class should have a method that can return the volume of the box. Create an object of the Box class and test the functionalities

OVERVIEW :

This experiment is based on the concept of Object-Oriented Programming in Java. A class named Box is created to represent a three-dimensional box. The class uses a parameterized constructor to initialize its dimensions — width, height, and depth. A method named volume() is used to calculate the volume of the box by multiplying these dimensions. In the main program, user input is taken to create a Box object and display the calculated volume. This practical helps in understanding how constructors and methods work together to perform operations on objects.

CODE :

```
import java.util.Scanner;
class Box{
    int width, height, depth;
    Box(int w, int h, int d){
        width = w;
        height = h;
        depth = d;
    }
    int volume() {
        return width * height * depth;
    }
}
public class exp1_1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the width, height and depth of the box:-");
        int w = sc.nextInt();
        int h = sc.nextInt();
        int d = sc.nextInt();
        Box b = new Box(w, h, d);
        System.out.println("Volume of the box is " + b.volume());
    }
}
```

OUTPUT :

Enter the width, height and depth of the box:-

10 5 2

Volume of the box is 100

EXPERIMENT - 2

AIM - Create a base class Fruit which has name ,taste and size as its attributes. A method called eat() is created which describes the name of the fruit and its taste. Inherit the same in 2 other class Apple and Orange and override the eat() method to represent each fruit taste. (Method overriding)

OVERVIEW :

This experiment demonstrates the concepts of inheritance and method overriding in Java. A base class named Fruit is created with attributes such as name, taste, and size, along with a method eat() that displays the fruit's name and taste. Two derived classes, Apple and Orange, inherit the properties of the Fruit class and override the eat() method to provide specific behavior for each fruit. The main program creates objects of Apple and Orange and calls their eat() methods to display their respective tastes. This practical helps in understanding how method overriding allows different classes to have their own implementations of the same method.

CODE :

```
class Fruit {
    String name, taste, size;
    void eat() {
        System.out.println("Name of the fruit is " + name + " and its taste is " + taste);
    }
}
class Apple extends Fruit {
    void eat() {
        System.out.println("Name of the fruit is " + name + " and its taste is " + taste);
    }
}
class Orange extends Fruit {
    void eat() {
        System.out.println("Name of the fruit is " + name + " and its taste is " + taste);
    }
}
public class exp1_2 {
    public static void main(String[] args) {
        Apple a = new Apple();
        a.name = "Apple";
        a.taste = "Sweet";
        a.eat();
        Orange o = new Orange();
        o.name = "Orange";
        o.taste = "Sour";
        o.eat();
    }
}
```

```
}  
}
```

OUTPUT :

```
Name of the fruit is Apple and its taste is Sweet  
Name of the fruit is Orange and its taste is Sour
```

EXPERIMENT - 3

AIM - Write a program to create a class named shape. It should contain 2 methods- draw() and erase() which should print “Drawing Shape” and “Erasing Shape” respectively. For this class we have three sub classes- Circle, Triangle and Square and each class override the parent class functions- draw () and erase (). The draw() method should print “Drawing Circle”, “Drawing Triangle”, “Drawing Square” respectively. The erase() method should print “Erasing Circle”, “Erasing Triangle”, “Erasing Square” respectively. Create objects of Circle, Triangle and Square in the following way and observe the polymorphic nature of the class by calling draw() and erase() methods using each object. Shape c=new Circle(); Shape t=new Triangle(); Shape s=new Square(); (Polymorphism)

OVERVIEW :

This experiment demonstrates the concept of polymorphism in Java using method overriding. A base class named Shape is created with two methods: draw() and erase(). Three subclasses—Circle, Triangle, and Square—inherit the Shape class and override these methods to provide their own specific implementations. In the main program, objects of Circle, Triangle, and Square are created using Shape class references. When the draw() and erase() methods are called, the appropriate subclass methods are executed, showing runtime polymorphism. This practical helps in understanding how a single reference can be used to call different behaviors based on the object it refers to.

CODE :

```
class Shape {
    void draw() {
        System.out.println("Drawing Shape");
    }
    void erase() {
        System.out.println("Erasing Shape");
    }
}
class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
    void erase() {
        System.out.println("Erasing Circle");
    }
}
class Triangle extends Shape {
```

```
void draw() {
    System.out.println("Drawing Triangle");
}

void erase() {
    System.out.println("Erasing Triangle");
}
}
class Square extends Shape {
    void draw() {
        System.out.println("Drawing Square");
    }
    void erase() {
        System.out.println("Erasing Square");
    }
}
class exp1_3 {
    public static void main(String[] args) {
        Shape c = new Circle();
        c.draw();
        c.erase();
        Shape t = new Triangle();
        t.draw();
        t.erase();
        Shape s = new Square();
        s.draw();
        s.erase();
    }
}
```

OUTPUT :

```
Drawing Circle
Erasing Circle
Drawing Triangle
Erasing Triangle
Drawing Square
Erasing Square
```

EXPERIMENT - 4

AIM - Write a Program to take care of Number Format Exception if user enters values other than integer for calculating average marks of 2 students. The name of the students and marks in 3 subjects are taken from the user while executing the program. In the same Program write your own Exception classes to take care of Negative values and values out of range (i.e. other than in the range of 0-100)

OVERVIEW :

This experiment demonstrates the concept of exception handling in Java. The program accepts the names and marks of two students in three subjects and calculates their average marks. It handles Number Format Exception when the user enters non-integer values and also uses user-defined exceptions to manage invalid inputs. A custom exception `NegativeValueException` is used when marks are less than zero, and `OutOfRangeException` is used when marks are greater than 100. By using try-catch blocks and custom exception classes, the program ensures that only valid data is processed, thereby improving the reliability and correctness of the program.

CODE :

```
import java.util.Scanner;
class NegativeValueException extends Exception {
    NegativeValueException(String s) {
        super(s);
    }
}
class OutOutOfRangeException extends Exception {
    OutOutOfRangeException(String s) {
        super(s);
    }
}
public class exp1_4 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] name = new String[2];
        int[][] marks = new int[2][3];
        int[] total = new int[2];
        for (int i = 0; i < 2; i++) {
            System.out.println("Enter the name of the student " + (i + 1));
            name[i] = sc.nextLine();
            sc.nextLine(); // Consume the newline character

            System.out.println("Enter the marks of the student " + (i + 1));
```

```

for (int j = 0; j < 3; j++) {
    try {
        marks[i][j] = sc.nextInt();
        if (marks[i][j] < 0)
            throw new NegativeValueException("Marks cannot be negative");
        if (marks[i][j] > 100)
            throw new OutOfRangeException("Marks should be in the range of 0-100");
    } catch (NegativeValueException e) {
        System.out.println(e);
    } catch (OutOfRangeException e) {
        System.out.println(e);
    }
    total[i] += marks[i][j];
}

System.out.println("Average marks of " + name[i] + " is " + total[i] / 3);
}
}
}

```

OUTPUT :

Enter the name of the student 1
yash

Enter the marks of the student 1
100
40
37

Average marks of yash is 59.0
Enter the name of the student 2
dev

Enter the marks of the student 2
10
-13

NegativeValueException: Marks cannot be negative
150

OutOfRangeException: Marks should be in the range of 0-100

EXPERIMENT - 5

AIM - Write a Java program to demonstrate the concept of multi-threading

OVERVIEW :

Multithreading is a feature of Java that allows a program to execute multiple tasks simultaneously. Each task runs as a separate thread within the same program. It helps in better CPU utilization and improves performance by performing multiple operations at the same time.

In multithreading, all threads share the same memory space but execute independently. It is commonly used in applications like games, web servers, and background processing.

Types of Thread Creation in Java

1) By Extending Thread Class

A class extends the Thread class and overrides the run() method. The thread starts execution when the start() method is called.

2) By Implementing Runnable Interface

A class implements the Runnable interface and defines the run() method. The object is passed to a Thread object and started using start().

CODE :

```
class MyThread extends Thread {
    public void run() {
        for(int i = 1; i <= 5; i++) {
            System.out.println("Thread is running: " + i);
        }
    }
}
```

```
public class MultiThreadDemo {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();

        for(int i = 1; i <= 5; i++) {
            System.out.println("Main thread: " + i);
        }
    }
}
```

OUTPUT :

```
Main thread: 1
Main thread: 2
Main thread: 3
Main thread: 4
Thread is running: 1
Main thread: 5
Thread is running: 2
Thread is running: 3
Thread is running: 4
Thread is running: 5
```

EXPERIMENT - 6

AIM - Implement Producer-Consumer Problem using multithreading.

OVERVIEW :

The Producer–Consumer problem is a classic example of process synchronization in multithreading. In this problem, one thread (Producer) produces data and another thread (Consumer) consumes it. Both share a common buffer.

To avoid conflicts such as consuming empty data or producing when the buffer is full, Java uses synchronization methods like:

- wait() – pauses a thread
- notify() – wakes up a waiting thread

This ensures proper coordination between Producer and Consumer.

CODE :

```
class SharedBuffer {
    int data;
    boolean hasData = false;

    synchronized void produce(int value) {
        while (hasData) {
            try {
                wait();
            } catch (Exception e) {}
        }

        data = value;
        System.out.println("Produced: " + data);
        hasData = true;
        notify();
    }

    synchronized void consume() {
        while (!hasData) {
            try {
                wait();
            } catch (Exception e) {}
        }

        System.out.println("Consumed: " + data);
        hasData = false;
        notify();
    }
}
```

```
}  
}
```

```
class Producer extends Thread {  
    SharedBuffer buffer;
```

```
    Producer(SharedBuffer b) {  
        buffer = b;  
    }
```

```
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            buffer.produce(i);  
        }  
    }  
}
```

```
class Consumer extends Thread {  
    SharedBuffer buffer;
```

```
    Consumer(SharedBuffer b) {  
        buffer = b;  
    }
```

```
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            buffer.consume();  
        }  
    }  
}
```

```
public class ProducerConsumerDemo {  
    public static void main(String[] args) {  
        SharedBuffer buffer = new SharedBuffer();
```

```
        Producer p = new Producer(buffer);  
        Consumer c = new Consumer(buffer);
```

```
        p.start();  
        c.start();  
    }  
}
```

OUTPUT :

Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5

EXPERIMENT - 7

AIM - Illustrate Deadlock in multithreading.

OVERVIEW :

Deadlock is a situation in multithreading where two or more threads are blocked forever, each waiting for a resource held by the other. As a result, none of the threads can continue execution. Deadlock usually occurs when:

- Thread 1 holds Resource A and waits for Resource B
- Thread 2 holds Resource B and waits for Resource A

Since both threads are waiting for each other, the program gets stuck.

CODE :

```
class DeadlockDemo {
    static final Object lock1 = new Object();
    static final Object lock2 = new Object();

    public static void main(String[] args) {

        Thread t1 = new Thread() {
            public void run() {
                synchronized (lock1) {
                    System.out.println("Thread 1: Holding lock1");

                    try { Thread.sleep(100); } catch (Exception e) {}

                    System.out.println("Thread 1: Waiting for lock2");
                    synchronized (lock2) {
                        System.out.println("Thread 1: Holding lock1 & lock2");
                    }
                }
            }
        };

        Thread t2 = new Thread() {
            public void run() {
                synchronized (lock2) {
                    System.out.println("Thread 2: Holding lock2");

                    try { Thread.sleep(100); } catch (Exception e) {}
```

```

        System.out.println("Thread 2: Waiting for lock1");
        synchronized (lock1) {
            System.out.println("Thread 2: Holding lock2 & lock1");
        }
    }
}
};

t1.start();
t2.start();
}
}

```

OUTPUT :

```

Thread 1: Holding lock1
Thread 2: Holding lock2
Thread 2: Waiting for lock1
Thread 1: Waiting for lock2

```

EXPERIMENT - 8

AIM - Write a Java program to demonstrate the concept of applet programming.

OVERVIEW :

Applet is a special type of program that is embedded in the webpage to generate the dynamic content. It runs inside the browser and works at client side.

Lifecycle of Java Applet

1. Applet is initialized.
2. Applet is started.
3. Applet is painted.
4. Applet is stopped.
5. Applet is destroyed.

java.applet.Applet class : provides 4 life cycle methods of applet.

1. **public void init():** is used to initialize the Applet. It is invoked only once.
2. **public void start():** is invoked after the init() method or browser is maximized. It is used to start the Applet.
3. **public void stop():** is used to stop the Applet. It is invoked when Applet is stop or browser is minimized.
4. **public void destroy():** is used to destroy the Applet. It is invoked only once.

java.awt.Component class : provides the following method:-

1. **public void paint(Graphics g):** is used to paint the Applet. It provides Graphics class object that can be used for drawing oval, rectangle, arc etc.

CODE :

```
class Fruit {
    String name, taste, size;
    void eat() {
        System.out.println("Name of the fruit is " + name + " and its taste is " + taste);
    }
}
class Apple extends Fruit {
    void eat() {
        System.out.println("Name of the fruit is " + name + " and its taste is " + taste);
    }
}
class Orange extends Fruit {
```



```
void eat() {  
    System.out.println("Name of the fruit is " + name + " and its taste is " + taste);  
}  
}  
public class exp1_2 {  
    public static void main(String[] args) {  
        Apple a = new Apple();  
        a.name = "Apple";  
        a.taste = "Sweet";  
        a.eat();  
        Orange o = new Orange();  
        o.name = "Orange";  
        o.taste = "Sour";  
        o.eat();  
    }  
}
```

OUTPUT :

Applet Programming

```
Name of the fruit is Apple and its taste is Sweet  
Name of the fruit is Orange and its taste is Sour
```

EXPERIMENT - 9

AIM - Write an applet for event handling which prints a message when clicked on the button.

OVERVIEW :

Java Applet event handling is a mechanism through which an applet responds to user interactions such as button clicks. It is implemented using the event delegation model, where a component (like a button) generates an event and a listener (such as ActionListener) handles it. When the button is clicked, the actionPerformed() method is invoked, and the applet updates the output using repaint() and paint() methods.

CODE :

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class ButtonApplet extends Applet implements ActionListener {

    Button btn;
    String message = "";

    public void init() {
        btn = new Button("Click Me");
        add(btn);

        btn.addActionListener(this);
    }

    public void actionPerformed(ActionEvent e) {
        message = "Button Clicked!";
        repaint();
    }

    public void paint(Graphics g) {
        g.drawString(message, 50, 100);
    }
}
```

HTML

```
<html>
<head>
    <title>Applet Event Handling</title>
</head>
```

```
<body>  
  <applet code="ButtonApplet.class" width="300" height="200">  
  </applet>  
</body>  
</html>
```

OUTPUT :

Event Handling via Applet

Click Me

Button Clicked!

EXPERIMENT - 10

AIM - Implement Painting using mouseDragged() method of MouseMotionListener in Applet

OVERVIEW :

Painting using mouseDragged() in a Java Applet is an example of mouse event handling. It is implemented using the MouseMotionListener interface, which detects mouse movement. When the user drags the mouse, the mouseDragged() method is repeatedly called, allowing the applet to draw shapes or lines at the current mouse coordinates. This creates a painting or drawing effect on the applet window.

CODE :

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;
public class JavaApplet extends Applet implements MouseMotionListener{
    public void init(){
        addMouseMotionListener(this);
        setBackground(Color.red);
    }
    public void mouseDragged(MouseEvent me){
        Graphics g=getGraphics();
        g.setColor(Color.white);
        g.fillOval(me.getX(),me.getY(),5,5);
    }
    public void mouseMoved(MouseEvent me){}
}
```

OUTPUT :

Painting using mouseDragged() method of MouseMotionListener in Applet



EXPERIMENT - 11

AIM - Write a Java program to demonstrate the concept of socket programming.

OVERVIEW :

Socket programming is a method used for communication between two computers over a network. It uses sockets as endpoints for sending and receiving data between a client and a server. In Java, sockets are implemented using classes like `Socket` and `ServerSocket`, enabling reliable communication using TCP protocol.

CODE :

Server Program

```
import java.io.*;
import java.net.*;

public class Server {
    public static void main(String[] args) {
        try {
            ServerSocket ss = new ServerSocket(5000);
            System.out.println("Server is waiting...");

            Socket s = ss.accept();
            System.out.println("Client connected");

            DataInputStream dis = new DataInputStream(s.getInputStream());
            String msg = dis.readUTF();

            System.out.println("Message from client: " + msg);

            ss.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Client Program

```
import java.io.*;
import java.net.*;

public class Client {
```

```
public static void main(String[] args) {  
    try {  
        Socket s = new Socket("localhost", 5000);  
  
        DataOutputStream dos = new DataOutputStream(s.getOutputStream());  
        dos.writeUTF("Hello Server!");  
  
        dos.flush();  
        dos.close();  
        s.close();  
    } catch (Exception e) {  
        System.out.println(e);  
    }  
}
```

OUTPUT :

A screenshot of a terminal window with a black background and white text. The text is as follows:
Server is waiting...
Client connected
Message from client:
Socket Programming

Hello Server!
The text is displayed in a monospaced font, typical of a terminal. There are some small, faint artifacts at the bottom of the window.

EXPERIMENT - 12

AIM - Implement Datagram UDP socket programming in java.

OVERVIEW :

UDP (User Datagram Protocol) is a connectionless communication protocol used for fast data transmission. In Java, it is implemented using DatagramSocket and DatagramPacket. Unlike TCP, UDP does not establish a connection and does not guarantee delivery, making it faster but less reliable. Data is sent in the form of packets called datagrams.

CODE :

Server Program

```
import java.net.*;

public class UDPServer {
    public static void main(String[] args) {
        try {
            DatagramSocket ds = new DatagramSocket(5000);
            byte[] receive = new byte[1024];

            DatagramPacket dp = new DatagramPacket(receive, receive.length);
            ds.receive(dp);

            String msg = new String(dp.getData(), 0, dp.getLength());
            System.out.println("Message from client: " + msg);

            ds.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Client Program

```
import java.net.*;

public class UDPClient {
    public static void main(String[] args) {
        try {
            DatagramSocket ds = new DatagramSocket();
```

```
String msg = "Hello UDP Server!";
byte[] send = msg.getBytes();

InetAddress ip = InetAddress.getByName("localhost");

DatagramPacket dp = new DatagramPacket(send, send.length, ip, 5000);
ds.send(dp);

ds.close();
} catch (Exception e) {
    System.out.println(e);
}
}
```

OUTPUT :

```
Datagram UDP Implementation
```

```
Message from client: Hello UDP Server!
```


EXPERIMENT - 13

AIM - Write a Java program to demonstrate the use of Java Beans.

OVERVIEW :

Java Beans are reusable software components written in Java that follow specific conventions such as having private properties, public getter and setter methods, and a no-argument constructor. They are used to encapsulate data and provide easy access to properties.

CODE :

StudentBean.java

```
import java.io.Serializable;

public class StudentBean implements Serializable {
    private int id;
    private String name;

    // No-argument constructor
    public StudentBean() {}

    // Getter and Setter methods
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

TestBean.java

```
public class TestBean {
    public static void main(String[] args) {
```

```
StudentBean s = new StudentBean();

s.setId(101);
s.setName("John");

System.out.println("JavaBean Usage \n");
System.out.println("ID: " + s.getId());
System.out.println("Name: " + s.getName());
}
}
```

OUTPUT :

JavaBean Usage

ID: 101

Name: John

EXPERIMENT - 14

AIM - Write a program in java to demonstrate encapsulation in java beans.

OVERVIEW :

Encapsulation is the process of wrapping data (variables) and code (methods) into a single unit and restricting direct access to the data. In Java Beans, this is achieved by declaring variables as private and accessing them through public getter and setter methods.

CODE :

EmployeeBean.java

```
import java.io.Serializable;

public class EmployeeBean implements Serializable {
    private int empId;
    private String name;
    private double salary;

    // No-argument constructor
    public EmployeeBean() {}

    // Getter and Setter methods
    public int getEmpId() {
        return empId;
    }

    public void setEmpId(int empId) {
        this.empId = empId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public double getSalary() {
        return salary;
    }
}
```

```
    public void setSalary(double salary) {  
        this.salary = salary;  
    }  
}
```

TestEncapsulation.java

```
public class TestEncapsulation {  
    public static void main(String[] args) {  
  
        EmployeeBean emp = new EmployeeBean();  
  
        emp.setEmpId(1);  
        emp.setName("Alice");  
        emp.setSalary(50000);  
  
        System.out.println("Encapsulation in Java Bean");  
        System.out.println("ID: " + emp.getEmpId());  
        System.out.println("Name: " + emp.getName());  
        System.out.println("Salary: " + emp.getSalary());  
    }  
}
```

OUTPUT :

```
Encapsulation in Java Bean  
ID: 1  
Name: Alice  
Salary: 50000.0
```

EXPERIMENT - 15

AIM - Create a servlet that recognizes a visitor for the first time to a web application and responds by saying “Welcome, you are visiting for the first time”. When the page is visited for the second time, it should say “Welcome Back”.

OVERVIEW :

Servlets can track users using cookies. When a user visits for the first time, a cookie is created and sent to the browser. On subsequent visits, the servlet checks for this cookie to identify returning users and responds accordingly.

CODE :

Servlet Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class VisitServlet extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        Cookie[] cookies = request.getCookies();
        boolean isNewUser = true;

        if (cookies != null) {
            for (Cookie c : cookies) {
                if (c.getName().equals("visited")) {
                    isNewUser = false;
                    break;
                }
            }
        }

        if (isNewUser) {
            out.println("<h2>Welcome, you are visiting for the first time</h2>");
            Cookie cookie = new Cookie("visited", "true");
            response.addCookie(cookie);
        } else {
```

```
        out.println("<h2>Welcome Back</h2>");
    }
}
```

web.xml code

```
<web-app>
  <servlet>
    <servlet-name>VisitServlet</servlet-name>
    <servlet-class>VisitServlet</servlet-class>
  </servlet>

  <servlet-mapping>
    <servlet-name>VisitServlet</servlet-name>
    <url-pattern>/visit</url-pattern>
  </servlet-mapping>
</web-app>
```

OUTPUT :

java servlet

Welcome, you are visiting for the first time

Welcome Back

EXPERIMENT - 15

AIM - Create a servlet that recognizes a visitor for the first time to a web application and responds by saying “Welcome, you are visiting for the first time”. When the page is visited for the second time, it should say “Welcome Back”.

OVERVIEW :

Servlets can track users using cookies. When a user visits for the first time, a cookie is created and sent to the browser. On subsequent visits, the servlet checks for this cookie to identify returning users and responds accordingly.

CODE :

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class VisitServlet extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        Cookie[] cookies = request.getCookies();
        boolean found = false;

        if (cookies != null) {
            for (Cookie c : cookies) {
                if (c.getName().equals("visited")) {
                    found = true;
                    break;
                }
            }
        }

        if (found) {
            out.println("Welcome Back");
        } else {
            Cookie c = new Cookie("visited", "true");
            response.addCookie(c);
            out.println("Welcome, you are visiting for the first time");
        }
    }
}
```

```
}  
}
```

OUTPUT :

java servlet

```
if ($?) { javac VisitServlet.java }; if ($?) { java VisitServlet }
```

```
Welcome, you are visiting for the first time
```


EXPERIMENT - 16

AIM - Write a program to set cookie information using Servlet.

OVERVIEW :

Cookies are small pieces of data stored on the client side. In a servlet, cookies are created using the Cookie class and sent to the browser using response.addCookie(). They help in storing user-specific information like name, preferences, etc.

CODE :

Servlet Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class SetCookieServlet extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Create cookies
        Cookie c1 = new Cookie("username", "John");
        Cookie c2 = new Cookie("course", "Java");

        // Set expiry time (optional)
        c1.setMaxAge(60*60); // 1 hour
        c2.setMaxAge(60*60);

        // Add cookies to response
        response.addCookie(c1);
        response.addCookie(c2);

        out.println("<h2>Cookies have been set!</h2>");
    }
}
```

web.xml code

```
<web-app>
  <servlet>
    <servlet-name>SetCookieServlet</servlet-name>
    <servlet-class>SetCookieServlet</servlet-class>
  </servlet>

  <servlet-mapping>
    <servlet-name>SetCookieServlet</servlet-name>
    <url-pattern>/setcookie</url-pattern>
  </servlet-mapping>
</web-app>
```

OUTPUT :

java servlet

Cookies have been set!

EXPERIMENT - 17

AIM - Write a Java program to show user validation using Servlet.

OVERVIEW :

Servlets are used to process client requests and perform server-side operations such as form handling and validation. User validation is an important aspect of web applications, ensuring that the data entered by users is correct and authorized.

In Servlets, user input is retrieved using request parameters through methods like `getParameter()`. The server then applies logical conditions to validate the input. Server-side validation is more secure than client-side validation, as it prevents unauthorized access and ensures data integrity before processing further.

CODE :

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class ValidationServlet extends HttpServlet {

    public void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String user = request.getParameter("username");
        String pass = request.getParameter("password");

        if (user.equals("admin") && pass.equals("1234")) {
            out.println("Login Successful");
        } else {
            out.println("Invalid Username or Password");
        }
    }
}
```

OUTPUT :

```
java prog> cd "c:\Users\hp\Desktop\coding\java prog"
\" ; if ($?) { javac student_A.java } ; if ($?) { java student_A }

Login Successful
```

EXPERIMENT - 18

AIM - Write a Java program to insert data into a table using JSP.

OVERVIEW :

JavaServer Pages (JSP) is a technology used to create dynamic web content by embedding Java code within HTML. It simplifies the development of web applications by allowing direct interaction with databases using JDBC.

JSP can be used to collect user input from forms and process it on the server side. Using JDBC (Java Database Connectivity), JSP can establish a connection with a database, execute SQL queries, and perform operations like insert, update, delete, and retrieve data. This enables dynamic data handling in web applications.

CODE :

index.jsp

```
<html>
<body>

<form action="insert.jsp" method="post">
    Name: <input type="text" name="name"><br>
    Age: <input type="text" name="age"><br>
    <input type="submit" value="Insert">
</form>

</body>
</html>
```

insert.jsp

```
<%@ page import="java.sql.*" %>
<html>
<body>

<%
    String name = request.getParameter("name");
    String age = request.getParameter("age");

    Class.forName("com.mysql.cj.jdbc.Driver");
    Connection con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/test", "root", ""
    );

    PreparedStatement ps = con.prepareStatement(
        "insert into student(name,age) values(?,?)"
    );
```

```
ps.setString(1, name);
ps.setInt(2, Integer.parseInt(age));

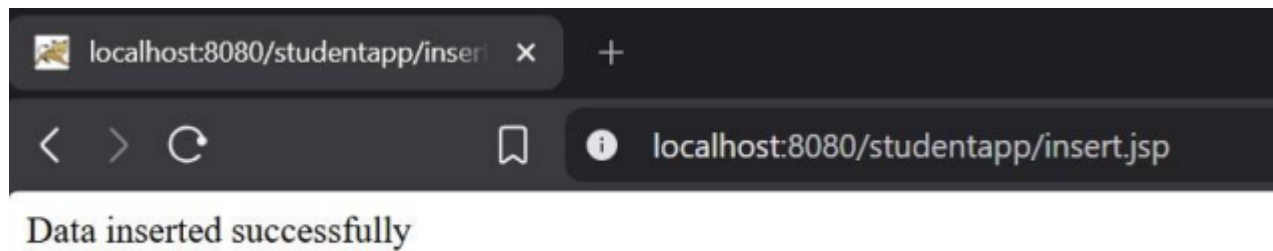
int i = ps.executeUpdate();

if (i > 0)
    out.println("Data inserted successfully");
else
    out.println("Insertion failed");

con.close();
%>

</body>
</html>
```

OUTPUT :



EXPERIMENT - 19

AIM - Write a JSP program to implement form data validation.

OVERVIEW :

JSP is used to handle user input and perform server-side validation in web applications. Form validation ensures that the data entered by the user meets required conditions before processing.

Validation can be done by retrieving form parameters using JSP implicit objects like request and applying logical checks. Server-side validation is important for security and data integrity, as it prevents invalid or malicious data from being processed by the application.

CODE :

form.jsp

```
<html>
<body>

<form action="validate.jsp" method="post">
    Username: <input type="text" name="username"><br>
    Password: <input type="password" name="password"><br>
    <input type="submit" value="Login">
</form>

</body>
</html>
```

validation.jsp

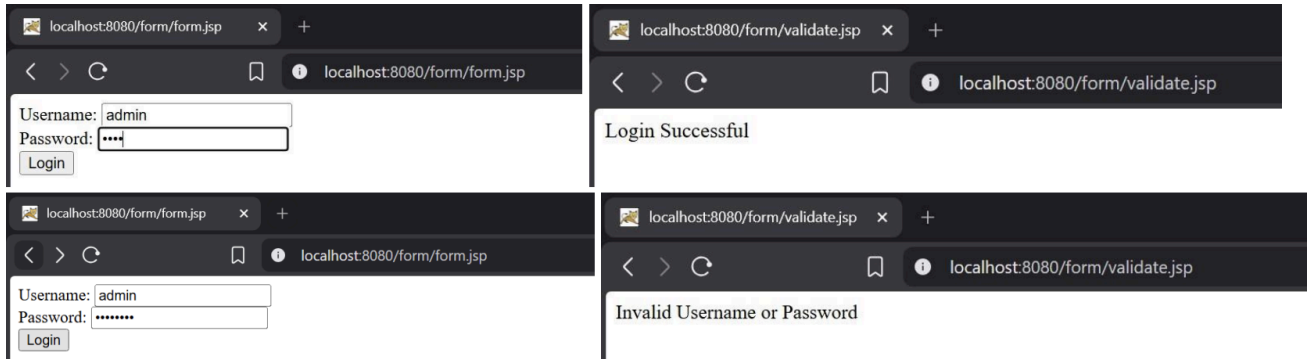
```
<html>
<body>

<%
    String user = request.getParameter("username");
    String pass = request.getParameter("password");

    if (user.equals("") || pass.equals("")) {
        out.println("Fields cannot be empty");
    } else if (user.equals("admin") && pass.equals("1234")) {
        out.println("Login Successful");
    } else {
        out.println("Invalid Username or Password");
    }
%>
```

```
</body>
</html>
```

OUTPUT :



EXPERIMENT - 20

AIM - Create a database in MySQL using JSP and perform insertion and retrieval operations

OVERVIEW :

JSP can be used to interact with MySQL databases using JDBC. It allows web applications to perform database operations like creating tables, inserting records, and retrieving data dynamically.

JDBC provides the required classes and methods to establish a connection with the database, execute SQL queries, and process results. Using JSP, both insertion and retrieval operations can be performed through user input and displayed dynamically on the web page.

CODE :

index.jsp

```
<html>
<body>
<form action="process.jsp" method="post">
    Name: <input type="text" name="name"><br>
    Age: <input type="text" name="age"><br>
    <input type="submit" value="Save">
</form>
<br>
<a href="display.jsp">View Records</a>
</body>
</html>
```

process.jsp

```
<%@ page import="java.sql.*" %>
<html>
<body>
<%
    String name = request.getParameter("name");
    String age = request.getParameter("age");

    Class.forName("com.mysql.cj.jdbc.Driver");
    Connection con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/test", "root", ""
    );

    Statement st = con.createStatement();
    st.executeUpdate("create table if not exists student(name varchar(50), age int)");

    PreparedStatement ps = con.prepareStatement("insert into student values(?,?)");
    ps.setString(1, name);
```



```
ps.setInt(2, Integer.parseInt(age));
ps.executeUpdate();
```

```
out.println("Data Inserted");
```

```
con.close();
%>
```

```
</body>
</html>
```

display.jsp

```
<%@ page import="java.sql.*" %>
```

```
<html>
```

```
<body>
```

```
<%
```

```
Class.forName("com.mysql.cj.jdbc.Driver");
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/test", "root", ""
);
```

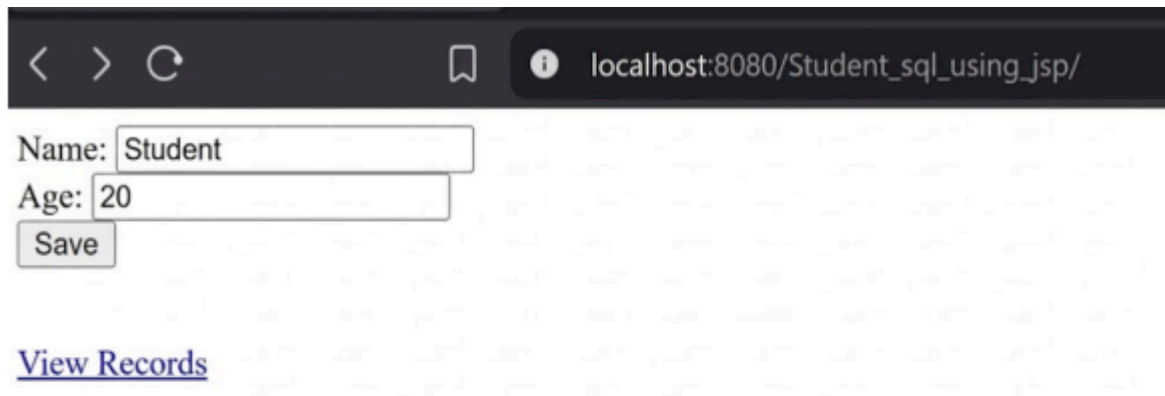
```
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("select * from student");
```

```
while (rs.next()) {
    out.println(rs.getString(1) + " " + rs.getInt(2) + "<br>");
}
```

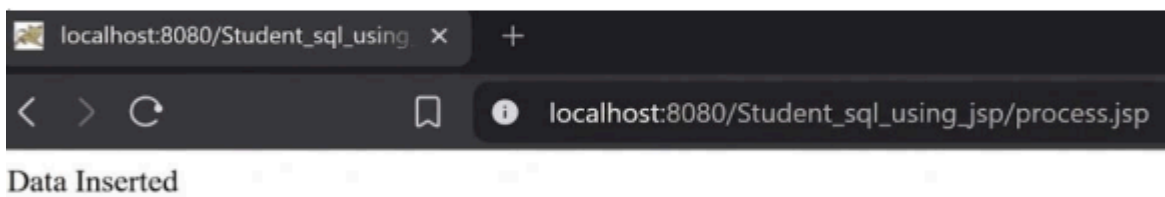
```
con.close();
%>
```

```
</body>
</html>
```

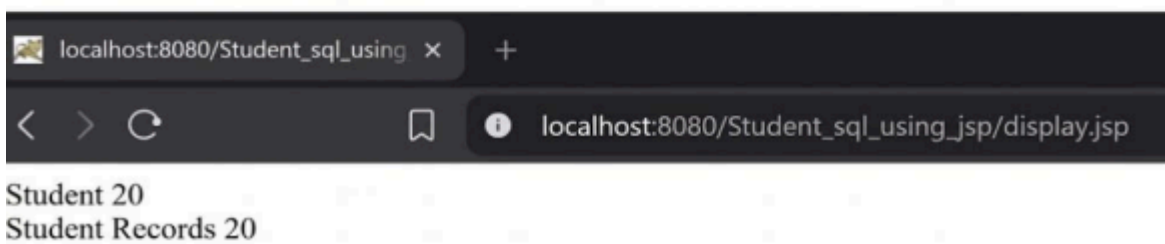
OUTPUT :



A screenshot of a web browser window. The address bar shows 'localhost:8080/Student_sql_using_jsp/'. The page contains a form with two input fields: 'Name:' with the value 'Student' and 'Age:' with the value '20'. Below these fields is a 'Save' button. At the bottom of the form, there is a blue underlined link that says 'View Records'.



A screenshot of a web browser window. The address bar shows 'localhost:8080/Student_sql_using_jsp/process.jsp'. The page displays the text 'Data Inserted' in a bold, black font.



A screenshot of a web browser window. The address bar shows 'localhost:8080/Student_sql_using_jsp/display.jsp'. The page displays two lines of text: 'Student 20' and 'Student Records 20'.

EXPERIMENT - 21

AIM - Create a Java JSP login and Sign Up form with Session using MySQL.

OVERVIEW :

JSP and Servlets use sessions to maintain user state across multiple requests. A session is created on the server and a unique session ID is assigned to each user. Using JSP with JDBC, user data can be stored in a MySQL database during sign-up and validated during login. Sessions are used to keep users logged in and manage authentication. This is widely used in web applications for secure user management.

CODE :

signup.jsp

```
<html>
<body>

<form action="register.jsp" method="post">
    Username: <input type="text" name="username"><br>
    Password: <input type="password" name="password"><br>
    <input type="submit" value="Sign Up">
</form>

</body>
</html>
```

register.jsp

```
<%@ page import="java.sql.*" %>
<html>
<body>

<%
    String user = request.getParameter("username");
    String pass = request.getParameter("password");

    Class.forName("com.mysql.cj.jdbc.Driver");
    Connection con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/test", "root", ""
    );

    Statement st = con.createStatement();
    st.executeUpdate(
        "create table if not exists users(username varchar(50), password varchar(50))"
    );
```

```

        PreparedStatement ps = con.prepareStatement("insert into users values(?,?)");
        ps.setString(1, user);
        ps.setString(2, pass);
        ps.executeUpdate();

        out.println("Registration Successful");

        con.close();
    %>

</body>
</html>

```

login.jsp

```

<html>
<body>

<form action="check.jsp" method="post">
    Username: <input type="text" name="username"><br>
    Password: <input type="password" name="password"><br>
    <input type="submit" value="Login">
</form>

</body>
</html>

```

check.jsp

```

<%@ page import="java.sql.*" %>
<html>
<body>

<%
    String user = request.getParameter("username");
    String pass = request.getParameter("password");

    Class.forName("com.mysql.cj.jdbc.Driver");
    Connection con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/test", "root", ""
    );

    PreparedStatement ps = con.prepareStatement(
        "select * from users where username=? and password=?"
    );

```

```
ps.setString(1, user);  
ps.setString(2, pass);
```

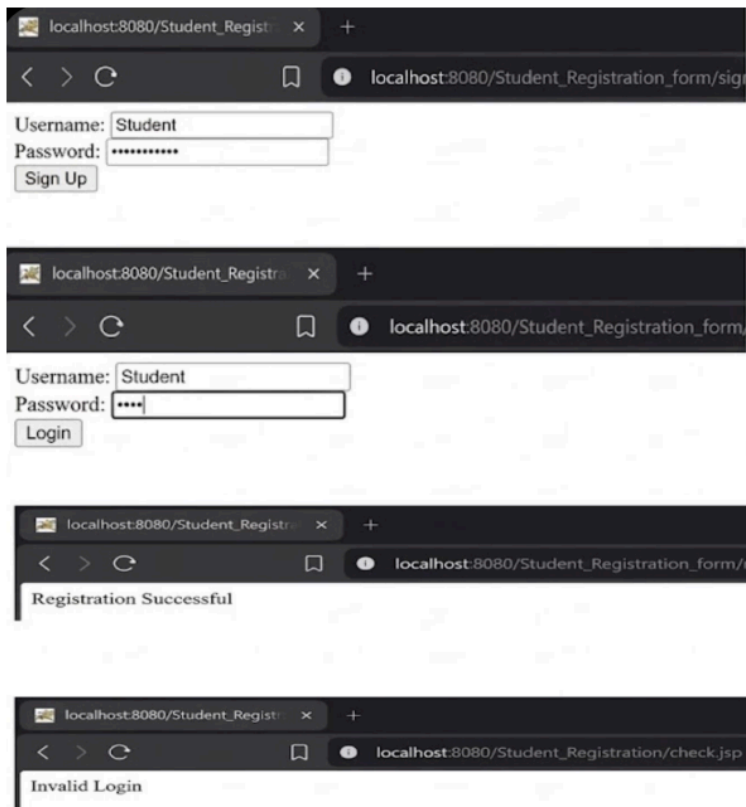
```
ResultSet rs = ps.executeQuery();
```

```
if (rs.next()) {  
    session.setAttribute("user", user);  
    out.println("Login Successful");  
} else {  
    out.println("Invalid Login");  
}
```

```
con.close();  
%>
```

```
</body>  
</html>
```

OUTPUT :



EXPERIMENT - 22

AIM - Create Employee Registration Form using a combination of JSP, Servlet, JDBC and MySQL database.

OVERVIEW :

JSP, Servlets, and JDBC are used together to build dynamic web applications with database connectivity. JSP is used to design the user interface and collect user input through forms. Servlets handle the business logic and process client requests.

JDBC (Java Database Connectivity) is used to connect the application with a MySQL database and execute SQL queries such as insert, update, and retrieval. This combination allows separation of presentation, processing, and data handling, making the application more structured and efficient.

CODE :

employee.jsp

```
<html>
<body>

<form action="register" method="post">
    Name: <input type="text" name="name"><br>
    Email: <input type="text" name="email"><br>
    Salary: <input type="text" name="salary"><br>
    <input type="submit" value="Register">
</form>

</body>
</html>
```

EmployeeServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.sql.*;

public class EmployeeServlet extends HttpServlet {

    public void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
```

```

String name = request.getParameter("name");
String email = request.getParameter("email");
String salary = request.getParameter("salary");

try {
    Class.forName("com.mysql.cj.jdbc.Driver");

    Connection con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/test", "root", ""
    );

    PreparedStatement ps = con.prepareStatement(
        "insert into employee(name,email,salary) values(?,?,?)"
    );

    ps.setString(1, name);
    ps.setString(2, email);
    ps.setDouble(3, Double.parseDouble(salary));

    int i = ps.executeUpdate();

    if (i > 0)
        out.println("Employee Registered Successfully");
    else
        out.println("Error in Registration");

    con.close();

} catch (Exception e) {
    out.println(e);
}
}

```

web.xml

```

<web-app>

    <servlet>
        <servlet-name>emp</servlet-name>
        <servlet-class>EmployeeServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>emp</servlet-name>
        <url-pattern>/register</url-pattern>
    </servlet-mapping>

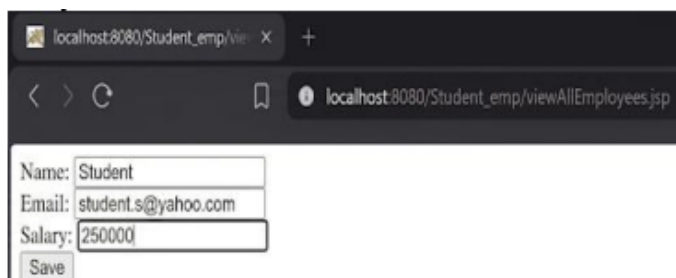
```

</web-app>

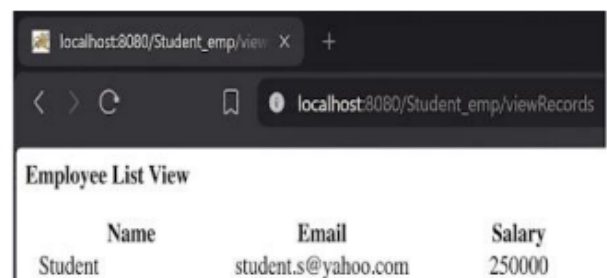
MySQL Table

```
CREATE TABLE employee (  
    name VARCHAR(50),  
    email VARCHAR(50),  
    salary DOUBLE  
);
```

OUTPUT :



A screenshot of a web browser window with the address bar showing 'localhost:8080/Student_emp/viewAllEmployees.jsp'. The page contains a form with three input fields: 'Name' with the value 'Student', 'Email' with the value 'student.s@yahoo.com', and 'Salary' with the value '250000'. Below the fields is a 'Save' button.



A screenshot of a web browser window with the address bar showing 'localhost:8080/Student_emp/viewRecords'. The page displays a table titled 'Employee List View' with three columns: 'Name', 'Email', and 'Salary'. The table contains one row with the values 'Student', 'student.s@yahoo.com', and '250000'.

Name	Email	Salary
Student	student.s@yahoo.com	250000

EXPERIMENT - 23

AIM - Develop a small web program using Servlets, JSPs with Database connectivity.

OVERVIEW :

Servlets, JSP, and JDBC are combined to create dynamic web applications with database interaction. JSP is used for designing the user interface and displaying data, while Servlets handle request processing and business logic.

JDBC is used to connect the application with a database and perform operations such as storing and retrieving data. This architecture follows a structured approach where JSP manages presentation, Servlet handles control logic, and JDBC manages database operations, making the application efficient and scalable.

CODE :

index.jsp

```
<html>
<body>

<form action="save" method="post">
    Name: <input type="text" name="name"><br>
    Course: <input type="text" name="course"><br>
    <input type="submit" value="Save">
</form>

<br>
<a href="view">View Students</a>

</body>
</html>
```

SaveServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.sql.*;

public class SaveServlet extends HttpServlet {

    public void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String name = request.getParameter("name");
```

```

String course = request.getParameter("course");

try {
    Class.forName("com.mysql.cj.jdbc.Driver");

    Connection con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/test", "root", ""
    );

    PreparedStatement ps = con.prepareStatement(
        "insert into student(name,course) values(?,?)"
    );

    ps.setString(1, name);
    ps.setString(2, course);
    ps.executeUpdate();

    response.sendRedirect("view");

    con.close();

} catch (Exception e) {
    PrintWriter out = response.getWriter();
    out.println(e);
}
}
}

```

ViewServlet.java

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.sql.*;

public class ViewServlet extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(

```

```

        "jdbc:mysql://localhost:3306/test", "root", ""
    );

    Statement st = con.createStatement();
    ResultSet rs = st.executeQuery("select * from student");

    while (rs.next()) {
        out.println(rs.getString("name") + " " + rs.getString("course") + "<br>");
    }

    con.close();

    } catch (Exception e) {
        out.println(e);
    }
}
}

```

web.xml

```

<web-app>

    <servlet>
        <servlet-name>save</servlet-name>
        <servlet-class>SaveServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>save</servlet-name>
        <url-pattern>/save</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>view</servlet-name>
        <servlet-class>ViewServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>view</servlet-name>
        <url-pattern>/view</url-pattern>
    </servlet-mapping>

</web-app>

```

MySQL Table

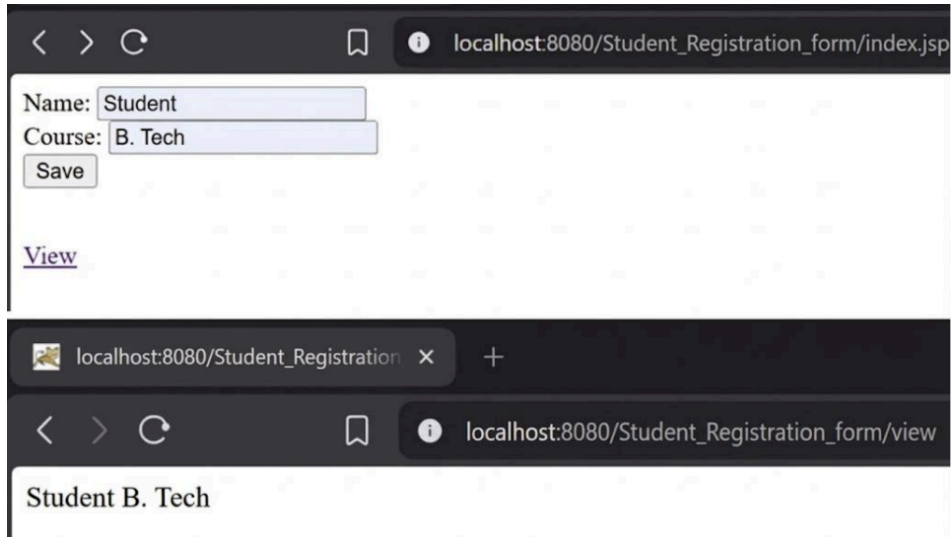
```

CREATE TABLE student (
    name VARCHAR(50),

```

```
course VARCHAR(50)
);
```

OUTPUT :



EXPERIMENT - 24

AIM - Create an RMI application where the server hosts a calculator service, and the client can perform basic arithmetic operations remotely

OVERVIEW :

Remote Method Invocation (RMI) is a Java API that allows a program to invoke methods on an object located on another machine. It enables distributed computing by allowing communication between client and server over a network.

In RMI, the server creates a remote object and registers it with the RMI registry. The client looks up this object and invokes methods remotely as if they were local. RMI uses interfaces, remote objects, and the registry to enable seamless communication between distributed systems.

CODE :

Calculator.java (Interface)

```
import java.rmi.*;
public interface Calculator extends Remote {

    int add(int a, int b) throws RemoteException;

    int sub(int a, int b) throws RemoteException;

    int mul(int a, int b) throws RemoteException;

    int div(int a, int b) throws RemoteException;
}
```

CalculatorImpl.java (Server Implementation)

```
import java.rmi.*;
import java.rmi.server.*;

public class CalculatorImpl extends UnicastRemoteObject implements Calculator {

    public CalculatorImpl() throws RemoteException {}

    public int add(int a, int b) throws RemoteException {
        return a + b;
    }
}
```

```

    public int sub(int a, int b) throws RemoteException {
        return a - b;
    }

    public int mul(int a, int b) throws RemoteException {
        return a * b;
    }

    public int div(int a, int b) throws RemoteException {
        return a / b;
    }
}

```

Server.java

```

import java.rmi.*;
import java.rmi.registry.*;

public class Server {

    public static void main(String[] args) {

        try {
            CalculatorImpl obj = new CalculatorImpl();

            LocateRegistry.createRegistry(1099);

            Naming.rebind("rmi://localhost:1099/calc", obj);

            System.out.println("Server Ready");

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

```

Client.java

```

import java.rmi.*;

public class Client {

    public static void main(String[] args) {

        try {

```

```
Calculator obj = (Calculator) Naming.lookup("rmi://localhost:1099/calc");
```

```
System.out.println("Addition: " + obj.add(10, 5));
```

```
System.out.println("Subtraction: " + obj.sub(10, 5));
```

```
System.out.println("Multiplication: " + obj.mul(10, 5));
```

```
System.out.println("Division: " + obj.div(10, 5));
```

```
    } catch (Exception e) {
```

```
        System.out.println(e);
```

```
    }
```

```
}
```

```
}
```

OUTPUT :

```
tudent_exp24> javac *.java
tudent_exp24> rmiregistry
ARNING: A terminally deprecated method in java.lang.System has been called
ARNING: System::setSecurityManager has been called by sun.rmi.registryImpl
ARNING: Please consider reporting this to the maintainers of sun.rmi.registry
ARNING: System::setSecurityManager will be removed in a future release
```

```
Student_exp24> java Client
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2
Student_exp24> □
```

EXPERIMENT - 25

AIM - Create a Hibernate application to perform CRUD (Create, Read, Update, Delete) operations on a Student entity.

OVERVIEW :

Hibernate is an Object Relational Mapping (ORM) framework that simplifies database interaction by mapping Java objects to database tables. It eliminates the need for writing complex SQL queries and provides methods to perform database operations directly using objects.

Hibernate uses configuration files and annotations/XML mapping to define the relationship between classes and tables. CRUD operations can be performed using Session and Transaction interfaces, making data handling efficient and reducing boilerplate code.

CODE :

Student.java

```
public class Student {

    private int id;
    private String name;
    private String course;

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCourse() {
        return course;
    }
}
```



```
public void setCourse(String course) {  
    this.course = course;  
}  
}
```

Student.hbm.xml

```
<!DOCTYPE hibernate-mapping PUBLIC  
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"  
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
```

```
<hibernate-mapping>  
    <class name="Student" table="student">  
  
        <id name="id" column="id">  
            <generator class="native"/>  
        </id>  
  
        <property name="name" column="name"/>  
        <property name="course" column="course"/>  
  
    </class>  
</hibernate-mapping>
```

hibernate.cfg.xml

```
<!DOCTYPE hibernate-configuration PUBLIC  
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"  
    "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">  
  
<hibernate-configuration>  
    <session-factory>  
  
        <property name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>  
        <property name="connection.url">jdbc:mysql://localhost:3306/test</property>  
        <property name="connection.username">root</property>  
        <property name="connection.password"></property>  
  
        <property name="dialect">org.hibernate.dialect.MySQL8Dialect</property>  
        <property name="hbm2ddl.auto">update</property>  
        <property name="show_sql">true</property>  
  
        <mapping resource="Student.hbm.xml"/>  
  
    </session-factory>  
</hibernate-configuration>
```

MainApp.java

```
import org.hibernate.*;
import org.hibernate.cfg.*;

public class MainApp {

    public static void main(String[] args) {

        SessionFactory factory = new Configuration().configure().buildSessionFactory();
        Session session = factory.openSession();

        Transaction t = session.beginTransaction();

        Student s = new Student();
        s.setName("Bhavya");
        s.setCourse("Java");

        session.save(s);
        t.commit();

        session.beginTransaction();

        Student s1 = session.get(Student.class, s.getId());
        System.out.println(s1.getName() + " " + s1.getCourse());

        session.getTransaction().commit();

        session.beginTransaction();

        s1.setCourse("Hibernate");
        session.update(s1);

        session.getTransaction().commit();

        session.beginTransaction();

        session.delete(s1);

        session.getTransaction().commit();

        session.close();
        factory.close();
    }
}
```

MySQL Table

```
CREATE TABLE student (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(50),  
    course VARCHAR(50)  
);
```

OUTPUT :

```
Hibernate: insert into student (course, name) values (?, ?)  
Record Inserted Successfully!  
  
Hibernate: select student0_.id as id1_0_0_, student0_.course as course2_0_0_, student0_.name as name3_0_0_ from student student0_  
Student Details: Student Java  
  
Hibernate: update student set course=?, name=? where id=?  
Record Updated Successfully!  
  
Hibernate: delete from student where id=?  
Record Deleted Successfully!
```